# SYSTEM ANALYSIS DOCUMENTATION (SAD)

## Team name: Sorite Gang

# 1. Introduction

## 1.1 Purpose

This System Analysis Documentation describes the technical scope, architecture, data flow, and operational design of the AI-Powered Corporate Blueprint Decision System. The system is built to support internal company decision-making by helping employees submit problem statements, generate AI-assisted blueprint proposals, review them across management levels, and track revisions through tickets.

The key elements covered in this document are:

- System architecture
- Data flow across users, services, databases, and AI components
- LLM service integration using Z.ai GLM API
- Core functional and non-functional requirements
- Monitoring, assumptions, dependencies, and project planning

## 1.2 Background

In many companies, idea evaluation and problem-solving discussions are handled manually through meetings, spreadsheets, chat messages, and email threads. This makes it difficult to standardize proposal quality, compare solutions fairly, and track feedback between departments.

To solve this, the proposed system introduces an AI-assisted workflow where Department Staff can submit a problem statement and upload company data, the system analyzes the submission using Z.ai GLM API, and generates several blueprint options. These blueprints are then reviewed by Department Leads and finally evaluated by the Director for approval, rejection, or merging.

This approach improves consistency, speeds up proposal preparation, and creates a structured review workflow across the organization.

## 1.3 Target Stakeholders

| Stakeholder | Role | Expectation |
|---|---|---|
| Department Staff | Submit problem statements, upload company data, review generated blueprints, and escalate selected blueprints | Easy submission flow, useful AI-generated options, clear feedback on revisions |
| Department Lead | Review escalated blueprints, classify by sector, validate recommendations, and | Clear comparison of blueprints, AI-assisted recommendation, efficient |

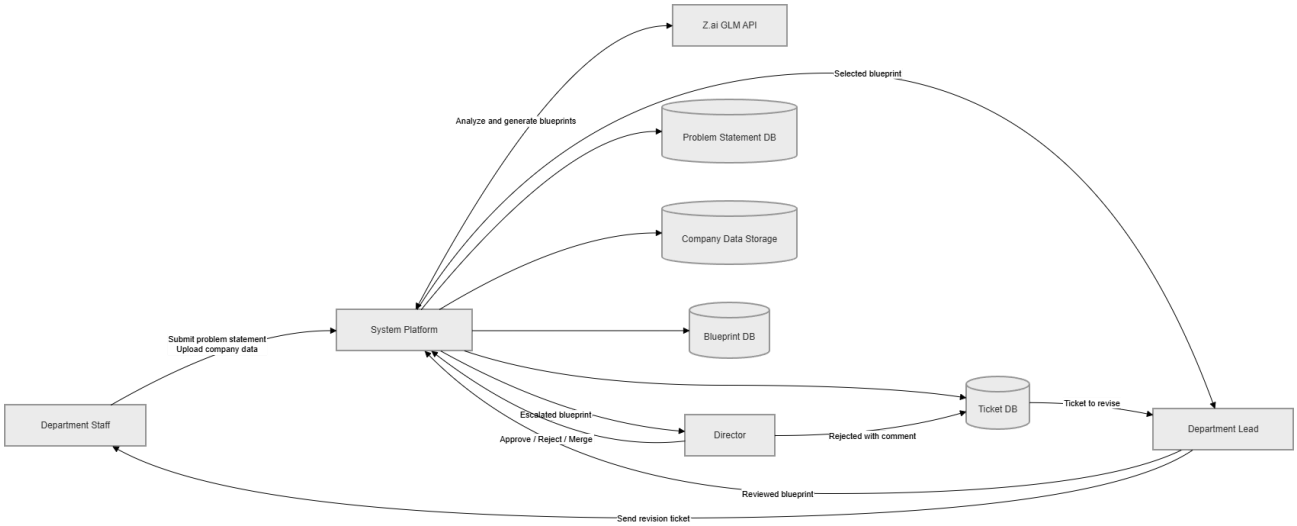
| | | |
|---|---|---|
| | escalate to director | review process |
| Director | Review final blueprints, approve, reject, or merge proposals | Fast and reliable decision support with complete review context |
| Development Team | Build and maintain the system | Modular design, clear service boundaries, manageable AI integration |
| QA Team | Test system behavior and workflow correctness | Clear process scope, testable inputs and outputs, traceable ticket flow |

# 2. System Architecture & Design

## 2.1 High-Level Architecture

The system follows a role-based client-server architecture with AI service integration. There are three primary user interfaces: Staff Portal, Lead Dashboard, and Director Dashboard. All interfaces communicate with a backend system that handles authentication, workflow management, storage, and AI orchestration.

The backend sends selected requests to the Z.ai GLM API for analysis, blueprint generation, blueprint classification, recommendation, and blueprint merging. The generated outputs are then stored and routed back to the correct user role.



### 2.1.1 Overview

| Type | Details |
|---|---|
| System | Web-based internal company platform |
| Architecture | Client-server architecture with AI service integration |
| Users | Department Staff, Department Lead, Director |
| External AI Service | Z.ai GLM API |
| Data Storage | Problem statements, uploaded company data, blueprints, tickets, audit logs |

The system consists of:

- User layer: Staff, Lead, Director
- Application layer: portals, dashboards, backend workflow system
- AI layer: prompt construction, analysis, blueprint generation, classification, recommendation, and merge logic
- Data layer: structured databases and document storage

### 2.1.2 Z.ai GLM Service Layer

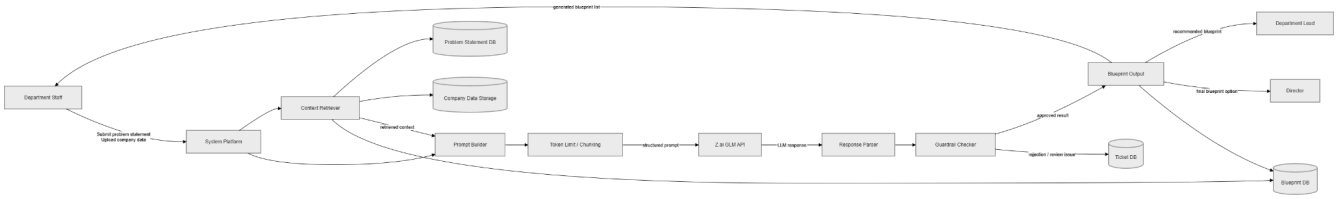
The system integrates Z.ai GLM API as a service layer instead of using a single generic AI module. The AI layer performs several focused tasks:

- Problem Analysis: analyzes the problem statement and uploaded company data
- Blueprint Generation: generates multiple blueprint options
- Classification: groups blueprints into sectors or categories
- Recommendation: identifies the most suitable blueprint from each sector
- Merge Support: combines selected blueprints into a new blueprint when requested by the director
- Ticket Assistance: helps structure review comments into revision tickets

The general AI flow is:

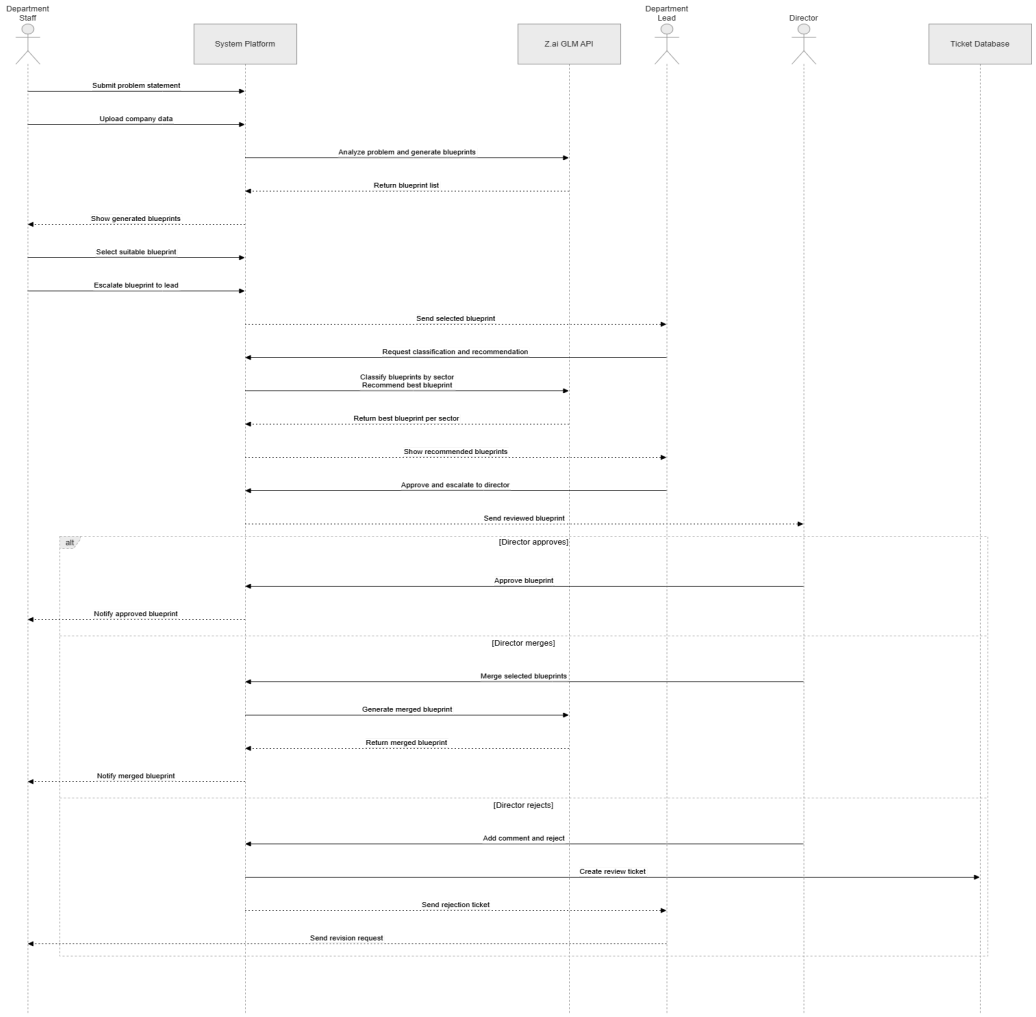
1. Staff submits a problem statement and optional company data
2. Backend retrieves relevant context and prepares the prompt
3. Prompt is checked for token size and chunked if needed
4. Request is sent to Z.ai GLM API

5. Response is parsed into structured blueprint results
6. Output is stored and displayed to the appropriate role




## 2.1.3 Sequence Diagram

The main sequence begins when Department Staff submit a problem statement and company data. The system forwards the data to Z.ai GLM API, receives blueprint results, and shows them to staff. The selected blueprint is escalated to Department Lead, then further reviewed and sent to the Director. The Director can approve, reject, or merge blueprints. Rejection triggers a ticket back to Lead and Staff for revision.



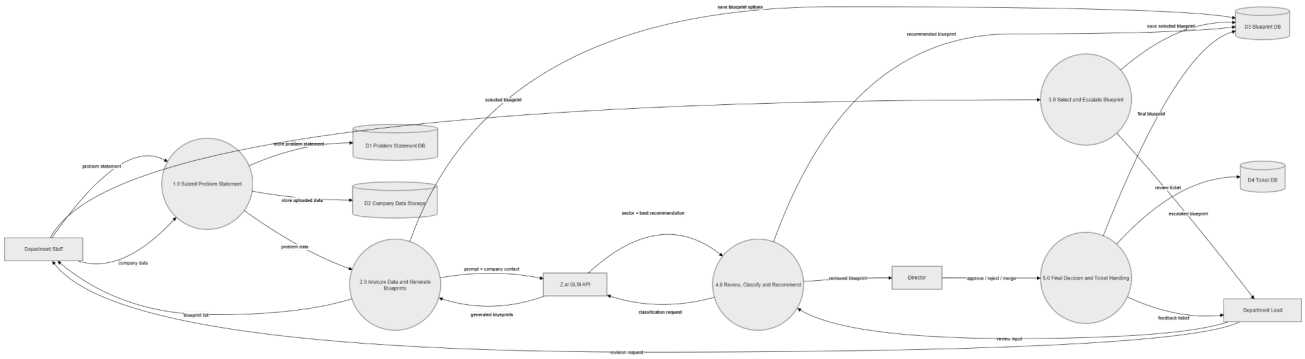
## 2.2 Technological Stack

| Layer | Proposed Technology |
|---|---|
| Frontend | React / Next.js |
| Backend | Node.js / Express or Next.js API routes |
| File Storage | Cloud object storage / secure document storage |
| AI Integration | Z.ai GLM API |
| Authentication | Role-based authentication and access control |
| Deployment | Cloud-hosted web deployment |

## 2.3 Key Data Flows

### 2.3.1 Data Flow Description

The main data flow begins when a staff member submits a problem statement and uploads company data. This information is stored and passed to the AI processing layer. The AI returns a list of generated blueprints. Staff then choose one blueprint and escalate it to Department Lead. The system performs classification and recommendation before sending shortlisted blueprints to the Director. The Director's final decision is stored in the blueprint database. If a revision is needed, a ticket is generated and sent back to the lower levels.



### 2.3.2 Normalized Database Schema

The system should use a normalized relational database design to maintain integrity and relationships between users, problem statements, blueprints, and tickets.

Suggested core entities:

- users
- problem_statements
- uploaded_documents
- blueprints
- blueprint_reviews
- tickets
- Audit_logs

Suggested relationships:

- One user can create many problem statements
- One problem statement can have many uploaded documents
- One problem statement can generate many blueprints
- One blueprint can go through many reviews
- One rejected blueprint can create one or more tickets
- One ticket can be linked to one blueprint and one reviewer comment chain

## 3. Functional Requirements & Scope

### 3.1 Minimum Viable Product (MVP)

| No. | Feature | Description |
|---|---|---|
| 1. | Problem Statement Submission | Department Staff can submit a new problem statement with title, description, and supporting context |
| 2. | Company Data Upload | Staff can upload company-related files or data for AI analysis |
| 3. | AI Blueprint Generation | Z.ai GLM API analyzes the submission and generates multiple blueprint options |
| 4. | Review and Escalation Workflow | Staff escalate selected blueprints to Lead, and Lead escalates reviewed blueprints to Director |

| | | |
|---|---|---|
| 5. | Decision and Ticket Handling | Director can approve, reject, or merge blueprints, and rejected items generate revision tickets |

### 3.2 Non-Functional Requirements (NFRs)

| Quality | Requirement | Implementation |
|---|---|---|
| 1. | Scalability | → System should support multiple departments and concurrent submissions<br>→ Modular backend services and separate storage for documents and records |
| 2. | Reliability | → Submitted data, blueprints, and tickets must not be lost during processing<br>→ Database persistence, validation, and audit logging |
| 3. | Maintainability | → Workflow logic should be easy to update as company review processes evolve<br>→ Clear separation of frontend, backend, AI orchestration, and database layers |
| 4. | Security | → Uploaded company data must only be visible to authorized roles<br>→ Role-based access control, authenticated APIs, and secure storage |
| 5. | AI Response Latency | → AI-generated blueprint responses should be returned within acceptable time under normal load<br>→ Prompt optimization, token control, loading states, and request timeouts |
| 6. | Cost Efficiency | → AI usage should remain within a controlled operational budget<br>→ Prompt length limits, chunking, caching, |

| | | |
|---|---|---|
| | | and restricting unnecessary repeat calls |

### 3.3 Out of Scope / Future Enhancements

- Real-time collaboration chat between Staff, Lead, and Director
- Advanced analytics dashboard for blueprint success trends
- Automatic integration with enterprise ERP or CRM systems
- Multilingual blueprint generation
- Mobile application support
- Full automated approval without human review

## 4. Monitoring, Evaluation, Assumptions & Dependencies

### 4.1 Technical Evaluation

#### 4.1.1 Gradual Rollout and Testing

The system should be deployed gradually. Early testing can begin with one department or a limited number of internal users. Once core functions are stable, usage can be expanded department by department. A/B testing can later be used to compare prompt strategies or blueprint presentation styles.

#### 4.1.2 Emergency Rollback and Recovery

If the system produces unstable outputs, routing errors, or workflow failures, the team should revert to the last stable release. A fallback mode can temporarily disable AI-generated recommendations and keep only manual submission and review workflow active.

#### 4.1.3 Priority Matrix

| Priority | Trigger Condition | Action |
|---|---|---|
| P1 Critical | System cannot generate or display blueprints | • Roll back AI workflow<br>• Switch to manual review mode |
| P2 High | Director or Lead cannot receive escalated blueprints | • Fix routing issue immediately |

| | | |
|---|---|---|
| | | ● Pause new escalations |
| P3 Medium | AI classification quality drops or responses are delayed | ● Tune prompts<br>● Reduce request size<br>● Retry with fallback logic |
| P4 Low | Minor UI issues or audit log display issues | ● Fix in the next release cycle |

## 4.2 Monitoring Plan

The system should monitor:

- AI request success and failure rate
- Average response time from Z.ai GLM API
- Number of blueprint generations per day
- Number of escalations from Staff to Lead and Lead to Director
- Number of rejected blueprints and generated tickets
- System errors in submission, file upload, and approval workflow

Monitoring alerts should be triggered when:

- AI requests repeatedly fail
- Ticket generation does not occur after rejection
- Escalated blueprints do not reach the next approver
- File uploads fail or cannot be retrieved

## 4.3 Assumptions

- Company users have stable internet access while using the system
- Staff upload valid and relevant company data
- Department Leads and Directors perform review actions in the system
- Z.ai GLM API remains available during normal operating hours
- Uploaded company documents are in supported file formats
- Users follow their assigned roles and permission boundaries

### 4.4 External Dependencies

| Tool / Service | Purpose | Risk |
|---|---|---|
| Z.ai GLM API | Core AI reasoning for:<br>● Analysis<br>● Generation<br>● Classification<br>● Recommendation<br>● Merge support | The system has a high risk if the AI API becomes unavailable or is rate-limited, as it is critical for core functionality. |
| Database Service | Stores structured data such as:<br>● Problem statements<br>● Blueprints<br>● Tickets | There is a medium risk if database connections or storage services fail, which may affect data access and system operations. |
| File Storage Service | ● Stores uploaded company data<br>● Stores supporting documents | There is a medium risk if documents cannot be uploaded or retrieved properly from the file storage system. |
| Authentication Service | ● Controls user login<br>● Manages role-based access | There is a high risk if user access control fails, potentially leading to security and authorization issues. |
| Notification Service | Sends alerts for:<br>● Escalation<br>● Rejection<br>● Revision actions | There is a low to medium risk if users do not receive action updates, which may impact workflow communication and user experience. |

| | | |
|---|---|---|

Mitigation strategies include:

- Retry logic for API calls
- Graceful fallback for AI outages
- Secure backups for structured data and uploaded files
- Audit logs for traceability
- Manual review fallback when AI is unavailable

# 5. Project Management & Team Contributions

## 5.1 Project Timeline

| Day | Activity |
|---|---|
| Day 1 | Project planning, requirement definition, and workflow design |
| Day 2 | High-level architecture, DFD, sequence diagram, and database design |
| Day 3–4 | Frontend prototype for Staff, Lead, and Director dashboards |
| Day 5–6 | Backend workflow implementation and database integration |
| Day 7 | Z.ai GLM API integration and prompt design |
| Day 8 | Ticketing and review workflow completion |
| Day 9 | Testing, debugging and refinement |
| Day 10 | Final demo preparation and documentation completion |

## 5.2 Team Member Roles

| Team Member | Role | Responsibility |
|---|---|---|
| Ng Xian Yang | Frontend Lead | Staff, Lead, and Director dashboard UI |
| Teoh Yu Heng | Backend Lead | Workflow APIs, escalation logic, and ticket handling |
| Mong Jing Yang | System Integration Lead | Deployment setup, system integration, monitoring, and performance optimization |
| Chen Yong Jeen | AI Integration Lead | Z.ai GLM API integration, prompt design, and response parsing |
| Teo Jun Ze | Database / QA Lead | Database schema, test cases, validation, and bug checking |

## 5.3 Recommendations

- Add caching for repeated analysis requests to reduce AI cost
- Add prompt templates for more consistent blueprint quality
- Add versioning for blueprints so changes can be traced clearly
- Add stronger analytics to measure approval rate, rejection rate, and time-to-decision
- Add document pre-processing for cleaner AI context extraction